

Final Report

ISYE6740: Computational Data Analysis

Name: Jacob Schroeder

Team Number: 42

Due Date: July 31, 2026

Contents

1	Introduction	3
2	Problem Statement	3
3	Data Source	4
4	Methodology	4
4.1	Preprocessing	4
4.2	Feature Engineering	4
4.3	Dimensionality Reduction for Visualization	6
4.4	Temporal Clustering	7
4.5	Robustness Check via DBSCAN	7
4.6	Drift Detection	8
4.7	Reproducibility	8
5	Results	8
5.1	Internal Clustering Validity	8
5.2	External Validation Against Theme Labels	8
6	Conclusion	9
A	Appendix	10
A.1	Rebrickable Entity Relationship Diagram	10
A.2	Breakdown of Number of Sets per Time Window	11
A.3	Additional UMAP Visualizations	12
	References	14

1. Introduction

LEGO has a fascinating history. Originating in Denmark in 1932, it began as a small wooden toy manufacturer. Since then it has grown into one of the most iconic toys ever produced. The LEGO brick as we know it today was introduced in 1958, and since then the brand has released thousands of distinct sets spanning dozens of licensed and original themes. Everything from the earliest Town and Castle series released in the 1970s, to the endlessly popular Star Wars licensed sets released beginning in the late 1990s, to the adult-targeted Icons and Architecture lines released for those of us who haven't fully grown up yet each set comes with hundreds of individual parts, colors, minifigures and other characteristics that make it both distinct and part of a greater collection.

Over the years, LEGO sets have evolved in piece count, colors, set complexity, and other key changes that make LEGO what they are. Things like the first lego sets with instructions being introduced in 1964, LEGO Technic introducing an entirely new style of piece getting added in 1977, the first Star Wars licensed set released in 1999, and robotics getting added with LEGO Mindstorms in 2006 have all been critical additions which have shaped the brand and the composition of those sets. This project examines that composition and how it has changed over the years.

2. Problem Statement

Can unsupervised machine learning detect meaningful structural shifts in LEGO set design over time, and do those detected shifts correspond to known historical events in the company's history?

This is the question that this project answers using temporal clustering [1] and distributional drift detection [3]. Over the past several decades, LEGO has expanded from a company focusing primarily on traditional building sets to a global entertainment brand encompassing licensed franchises, collector-focused products, and highly specialized themes. These changes are incredibly well documented from a business perspective, but it can be very unclear how LEGO's product designs have evolved over time when viewed solely through the characteristics of the sets.

The objective of this project is to analyze the evolution of LEGO products from the 1950s through the present day using unsupervised machine learning techniques. Specifically, LEGO sets will be grouped into clusters based on structural and design-related features and the analysis examines how these clusters change over time. When looking at the shifts in the composition of clusters across different years, the expectation is that certain substantial changes will appear aligning with major changes to LEGO's product portfolio.

Let $S = \{s_1, s_2, \dots, s_N\}$ denote the full catalog of official LEGO sets, those being the sets released for standard sale and not including any of the promotional materials, special editions, or subsets which are sold as larger groups. Each set s_i is characterized by a feature vector $\mathbf{x}_i \in \mathbb{R}^d$ derived from its structural composition. Each set carries a release year $y_i \in \{1955, \dots, 2026\}$. We partition S into cohorts made up of one or more years, $C_t = \{s_i : y_i = t\}$ for each time period t .

There are two ideas at the core of this problem:

1. **Temporal Clustering:** For each cohort C_t , this project fits a clustering model K_t that partitions sets into k groups based on structural similarity. The cluster centroids are tracked $\{\mu_1^{(t)}, \dots, \mu_k^{(t)}\}$ for each period t .
2. **Drift Detection:** Measuring the degree of distributional change between consecutive cohort clusterings K_t and K_{t+1} using a quantitative dissimilarity metric. This will identify specific

years, t^* , where this dissimilarity exceeds a threshold. These years will be noted as *drift points*.

A formal notation of the goal of this project is to estimate:

$$\Delta_t = \text{drift}(K_t, K_{t+1}), \quad t \in \{1955, \dots, 2026\}$$

where $\text{drift}(K_t, K_{t+1})$ is the drift shown by clustering dissimilarity measured using centroid displacement and Adjusted Rand Index. A detected drift point is defined as:

$$t^* = \{t : \Delta_t > \mu_\Delta + c\sigma_\Delta\}$$

c is a tunable threshold and μ_Δ and σ_Δ are the mean and standard deviation of Δ_t across all years.

The hypothesis is that drift points t^* will cluster near known historical milestones in LEGO's product history. Some examples of this would be the introduction of the Star Wars license in 1999, the near bankruptcy of the company in 2003 – 2004, the growth of licensed themes throughout the 2010s, and the growth of adult targetted sets in the 2020s. If these events can be detected using structural, label-free features this would constitute a meaningful unsupervised result.

3. Data Source

Data has been sourced from the Rebrickable database [7], which provides both API and CSV exports for all lego set related data. For this project, I used the CSV export of a large relational database including the following tables: `sets`, `themes`, `parts`, `part_categories`, `colors`, `inventories`, `inventory_parts`, `inventory_minifigs`, and `minifigs`. The complete entity relationship diagram including all attributes can be found in Appendix A.1. These were loaded into a local SQLite database via Python's `sqlite3` module and queried using `pandas`.

The complete dataset is 149.3 MB containing over 21,500 individual lego sets.

4. Methodology

4.1 Preprocessing

Sets with a very small number ($n < 20$) of parts were excluded as well as non-standard releases (e.g., promotional packs, individual minifigures, individual parts sold as set, merchandise with set numbers). After preprocessing, I am left with just over 12,000 standard lego sets. Additionally, years with a small number of sets released in the catalog were grouped using rolling windows to ensure a large enough cohort size for stable clustering. Grouping was accomplished by setting a minimum cohort size (100) along with a maximum number of years per cohort (12). This ensured sufficient cohort size while not allowing for too many years to be clustered. Years with more than the minimum cohort size were evaluated alone. A detailed breakdown of the number of sets included in each cohort can be found in Appendix A.2.

4.2 Feature Engineering

Each set s_i is represented by a d -dimensional feature vector spanning four categories: scale (total part count, unique part count, minifigure count, spare part ratio), diversity (Shannon entropy

of part categories H_{cat} , Shannon entropy of colors H_{col} , number of distinct colors, number of distinct part categories), composition (proportion of Technic parts, proportion of decorative/printed parts, proportion of transparent parts via `colors.is_trans`, rare part ratio for parts appearing in fewer than 20 sets), and minifigure characteristics (minifigure-to-part ratio, a binary indicator for minifigure presence).

Before modeling, I screened this candidate set for multicollinearity using Variance Inflation Factors (Table 1). Two features stood out immediately: `total_part_count` and `num_parts` had VIF values above 7000, reflecting their near-total redundancy, and `n_distinct_categories` exceeded the common threshold of 10. We removed both `total_part_count` and `n_distinct_categories` and recomputed VIF on the remaining twelve features (Table 2); all values fell below 5, indicating the reduced set carries largely independent information.

Table 1: Variance Inflation Factors (Original Feature Set)

#	Feature	VIF
0	num_parts	8299.992733
1	total_part_count	8145.789045
2	unique_part_count	13.160328
3	minifig_count	5.305488
4	spare_part_ratio	1.297114
5	h_cat	5.350565
6	h_col	3.749628
7	n_distinct_colors	7.882505
8	n_distinct_categories	15.857922
9	prop_technic	1.231668
10	prop_printed	1.929013
11	prop_transparent	1.186414
12	rare_part_ratio	2.373186
13	minifig_to_part_ratio	1.317496
14	has_minifigs	2.428044

Table 2: Variance Inflation Factors (Reduced Feature Set)

#	Feature	VIF
0	num_parts	2.904824
1	unique_part_count	4.509105
2	minifig_count	1.952869
3	spare_part_ratio	1.245666
4	h_cat	2.159906
5	h_col	1.521950
6	prop_technic	1.224180
7	prop_printed	1.924807
8	prop_transparent	1.185894
9	rare_part_ratio	2.363765
10	minifig_to_part_ratio	1.286617
11	has_minifigs	2.123624

The retained twelve features still span the same three functional roles: scale and part diversity

(`num_parts`, `unique_part_count`), minifigure presence and intensity (`minifig_count`, `has_minifigs`, `minifig_to_part_ratio`), and material or stylistic composition (`h_cat`, `h_col`, `prop_technic`, `prop_printed`, `prop_transparent`, `rare_part_ratio`, `spare_part_ratio`). Rather than relying on piece count alone, this combination captures how a set is built. Each feature is normalized prior to clustering. Note that this VIF-reduced set is used only for the UMAP visualization described in Section 4.3; the temporal clustering described in Section 4.4 uses the full feature set, since collinearity among features is less consequential for distance-based clustering methods like k -means than it is for interpreting a low-dimensional projection.

4.3 Dimensionality Reduction for Visualization

To visualize the structure of the feature space described above, I applied Uniform Manifold Approximation and Projection (UMAP) [5] to the normalized feature matrix. UMAP was chosen over alternatives such as PCA or t -SNE for its ability to preserve both local neighborhood structure and, to a reasonable extent, global relationships between clusters, while remaining computationally efficient on the dataset size. I used a neighborhood size of $n_{\text{neighbors}} = 15$ and minimum distance of 0.1, projecting the data into two components for the primary visualization and three components as a check on whether additional structure emerges beyond what is visible in two dimensions.

Figure 1 shows the resulting two-dimensional embedding, with points colored by time window to indicate when each set was released. The embedding reveals three to four well-separated clusters, along with a small number of isolated outlier points at the periphery. This separation indicates that the feature set captures genuinely distinct regimes of LEGO sets, likely corresponding to differences in theme, set type, or piece composition, that do not blend continuously into one another.

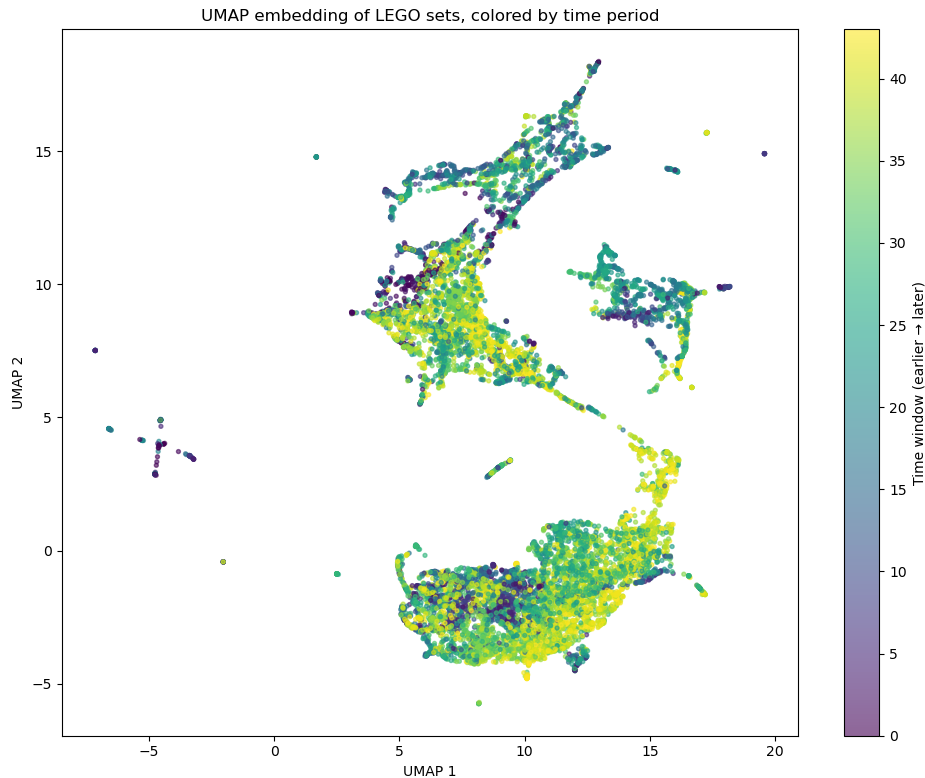


Figure 1: Two-dimensional UMAP embedding of LEGO sets, colored by time window.

Notably, time window color is thoroughly mixed within each cluster rather than segregated: early and late sets of a given type occupy the same region of the embedding. This suggests that time of release is not the dominant axis separating set types, since a strong time effect would appear as solid-colored blobs rather than mixed ones. A weaker secondary time trend is still visible, however: later sets tend to concentrate toward the outer edges of each cluster, while earlier sets sit closer to the center, implying that set characteristics drift gradually over time within a given type even though type itself is the primary driver of cluster membership.

The embedding also exhibits several thin tendrils connecting or extending from the main clusters. These likely correspond to sets that are transitional between two categories, or to sparsely represented set variants such as reissues. Identifying which sets populate these bridging regions is a natural next step for validating the semantic meaning of the clusters.

To confirm that two dimensions are sufficient to capture the dominant structure, I repeated the embedding with three components and inspected it from multiple viewing angles (Figure 4 in Appendix A.3). The three-dimensional projection reproduces the same three-cluster structure and the same within-cluster mixing of time windows observed in two dimensions, with no additional cluster or separation emerging along the third axis. This indicates that the two-dimensional embedding is not discarding meaningful structure, and that two components are adequate for downstream visualization and interpretation.

4.4 Temporal Clustering

The optimal number of clusters k is selected using the elbow method and silhouette score analysis on the second-to-last cohort rather than the most recent one. The final cohort in the dataset does not correspond to a complete year, since it captures only the partial set of releases available at the time of data collection, whereas the second-to-last cohort corresponds to 2025, the most recent complete year of releases. Selecting k from a full year avoids anchoring the clustering solution to an incomplete and potentially unrepresentative sample. Cluster fitting itself proceeds backward in time, beginning from this reference cohort and moving through earlier cohorts, so that each earlier cohort’s structure is interpreted relative to present-day design conventions rather than relative to the earliest sets in the catalog. Drift, by contrast, is calculated moving forward through time: Δ_t measures the change from cohort t to cohort $t + 1$, so that detected drift points t^* correspond to the year in which a shift occurred, consistent with the historical framing in Section 2.

To address any concern that standard k -means clustering applied independently per cohort may produce inconsistent labeling (the “cluster correspondence problem”), centroids were matched across consecutive years using the Hungarian algorithm to minimize total centroid displacement [4].

4.5 Robustness Check via DBSCAN

As a robustness check on the k -means-based temporal clustering, I will additionally fit DBSCAN [2] to each cohort $\mathcal{C}_t$. Unlike k -means, DBSCAN does not require a fixed number of clusters to be specified in advance and can identify clusters of arbitrary shape, while explicitly labeling low-density points as noise rather than forcing them into the nearest cluster. This makes it a useful check on two assumptions underlying the primary k -means approach: first, that the fixed value of k selected from the 2025 reference cohort remains an appropriate choice across earlier cohorts with potentially different underlying structure, and second, that LEGO sets naturally form roughly spherical, similarly sized groups in feature space, an assumption implicit in k -means but not required by DBSCAN. Rather than treating DBSCAN as a competing model to be scored against k -means,

its purpose here is diagnostic: if DBSCAN, run independently and without the constraint of a fixed k , recovers a broadly similar number and shape of clusters within each cohort, this lends confidence that the cluster structure detected by k -means reflects genuine groupings in the data rather than an artifact of its shape assumptions. If DBSCAN instead reveals substantially different structure, particularly within cohorts flagged as drift points, this would signal that conclusions drawn from the k -means clustering warrant additional caution rather than invalidating them outright.

4.6 Drift Detection

The dissimilarity between consecutive clustering solutions were measured using two metrics:

1. **Adjusted Rand Index (ARI):** For sets present in both cohorts $\mathcal{C}_t$ and $\mathcal{C}_{t+1}$ (overlapping years), ARI will measure the agreement between the two clustering assignments. A sharp drop in ARI signals structural reorganization [6].
2. **Centroid displacement:** The average Euclidean distance between matched centroid pairs $\|\mu_j^{(t+1)} - \mu_j^{(t)}\|_2$, averaged over all k clusters, captures smooth directional drift even when set membership does not overlap between cohorts.

4.7 Reproducibility

All code has been written in Python and made available alongside the SQLite database construction scripts. Random seeds were fixed at 42 (the meaning of life, the universe, and everything) to ensure reproducible random cluster initializations.

5. Results

5.1 Internal Clustering Validity

For each annual clustering solution $\mathcal{K}_t$, internal quality will be assessed using:

- **Silhouette score:** Measures how similar each point is to its own cluster relative to other clusters; scores near 1 indicate well defined clusters [8].
- **Davies-Bouldin Index:** Penalizes clusters that are large and close together; lower values indicate better separation.
- **Elbow method on inertia:** Confirms that the chosen k is appropriate and stable across years.

Consistency of these metrics across years will itself be informative: a sudden degradation in silhouette score in a given year suggests the data no longer conforms to the assumed cluster geometry, which is an independent signal of drift.

5.2 External Validation Against Theme Labels

Although theme labels are not used as features, they can be used for external validations. After fitting each annual clustering, I will compute:

- **Normalized Mutual Information (NMI)** between cluster assignments and official theme labels

- **Homogeneity and completeness scores** to assess whether clusters align with the theme.

6. Conclusion

LEGO has long been a passion of mine from a very young age. While my appreciation for the brand and the product began as a child, it has remained a hobby well into adulthood. My growth reflects LEGO's growth. When I was younger, I thought that LEGO Star Wars was the greatest thing on earth. While I still love those sets, my passions have shifted towards the large-scale car models like the Formula 1 cars, Porsche 911, Land Rover Defender, as well as the new LEGO Lord of the Rings collection.

This project gives me a great opportunity to combine a personal passion for LEGO with the machine learning techniques studied in this course. By applying clustering and drift detection methods to decades of LEGO set data, I hope to uncover patterns in the evolution of the brand and show how the composition of the product design which has made this iconic brand so successful reveals shifts not obvious to the human eye.

A. Appendix

A.1 Rebrickable Entity Relationship Diagram

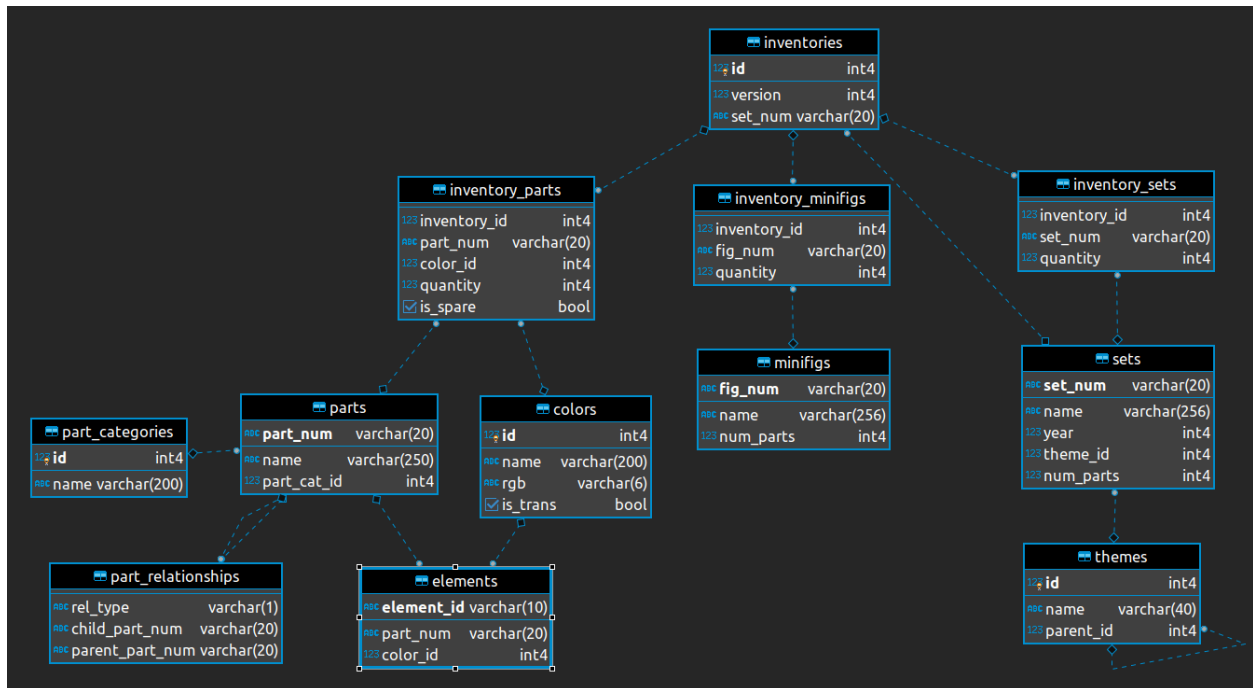


Figure 2: Database schema for the Rebrickable [7] LEGO dataset. The relational structure links LEGO sets to inventories, parts, colors, themes, and minifigures, providing the foundation for feature engineering and longitudinal analysis of changes in LEGO product design over time.

A.2 Breakdown of Number of Sets per Time Window

Table 3: Number of Sets per Time Window

Window	num sets	min	max	Window	num sets	min	max
0	56	1955	1966	22	227	2005	2005
1	131	1967	1973	23	216	2006	2006
2	110	1974	1976	24	209	2007	2007
3	149	1977	1979	25	238	2008	2008
4	101	1980	1981	26	272	2009	2009
5	137	1982	1984	27	252	2010	2010
6	104	1985	1985	28	311	2011	2011
7	219	1986	1987	29	358	2012	2012
8	134	1988	1989	30	362	2013	2013
9	178	1990	1991	31	423	2014	2014
10	179	1992	1993	32	471	2015	2015
11	105	1994	1994	33	496	2016	2016
12	130	1995	1995	34	475	2017	2017
13	139	1996	1996	35	487	2018	2018
14	181	1997	1997	36	479	2019	2019
15	276	1998	1998	37	496	2020	2020
16	230	1999	1999	38	521	2021	2021
17	265	2000	2000	39	508	2022	2022
18	232	2001	2001	40	532	2023	2023
19	250	2002	2002	41	610	2024	2024
20	269	2003	2003	42	640	2025	2025
21	250	2004	2004	43	452	2026	2026

Number of sets per time window. Each window aggregates one or more consecutive release years to balance sample sizes across the dataset's history; **Window** is the window index, **num sets** is the count of LEGO sets falling in that window, and **min/max** give the first and last release year included in the window.

A.3 Additional UMAP Visualizations

This appendix presents supplementary UMAP visualizations that support the discussion in Section 4.3 (dimensionality reduction for visualization).



Figure 3: Two-dimensional UMAP embedding faceted by time window. In each panel, points from that window are highlighted in color and all other points are shown in gray for reference.

Figure 3 shows the same two-dimensional embedding as Figure 1, but faceted by individual time window rather than using a single continuous color scale. In each panel, points belonging to that window are highlighted in color while all other points are shown in gray for reference. This view makes the temporal drift within clusters easier to inspect than a single overlaid plot. In the earliest windows (0 through roughly 5), highlighted points are concentrated almost entirely in a small region of the upper cluster, indicating that sets released in this period are compositionally similar to one another and represent a narrow slice of the overall feature space. Moving through the middle

windows, the highlighted region expands and begins to populate the lower cluster as well, suggesting the introduction of new set types over time rather than a simple continuation of earlier patterns. By the later windows (roughly 30 onward), highlighted points spread across nearly the full extent of both major clusters, indicating that recent sets span a much broader range of the feature space than earlier ones. This progressive broadening is consistent with the interpretation in Section 4.3 that set diversity, in terms of scale, composition, and minifigure content, has increased over time, even though time window itself is not the primary axis separating the clusters.

Figure 4 shows the three-dimensional UMAP embedding from multiple viewing angles, used to confirm that no additional cluster structure is hidden along a third dimension beyond what is visible in the two-dimensional embedding.

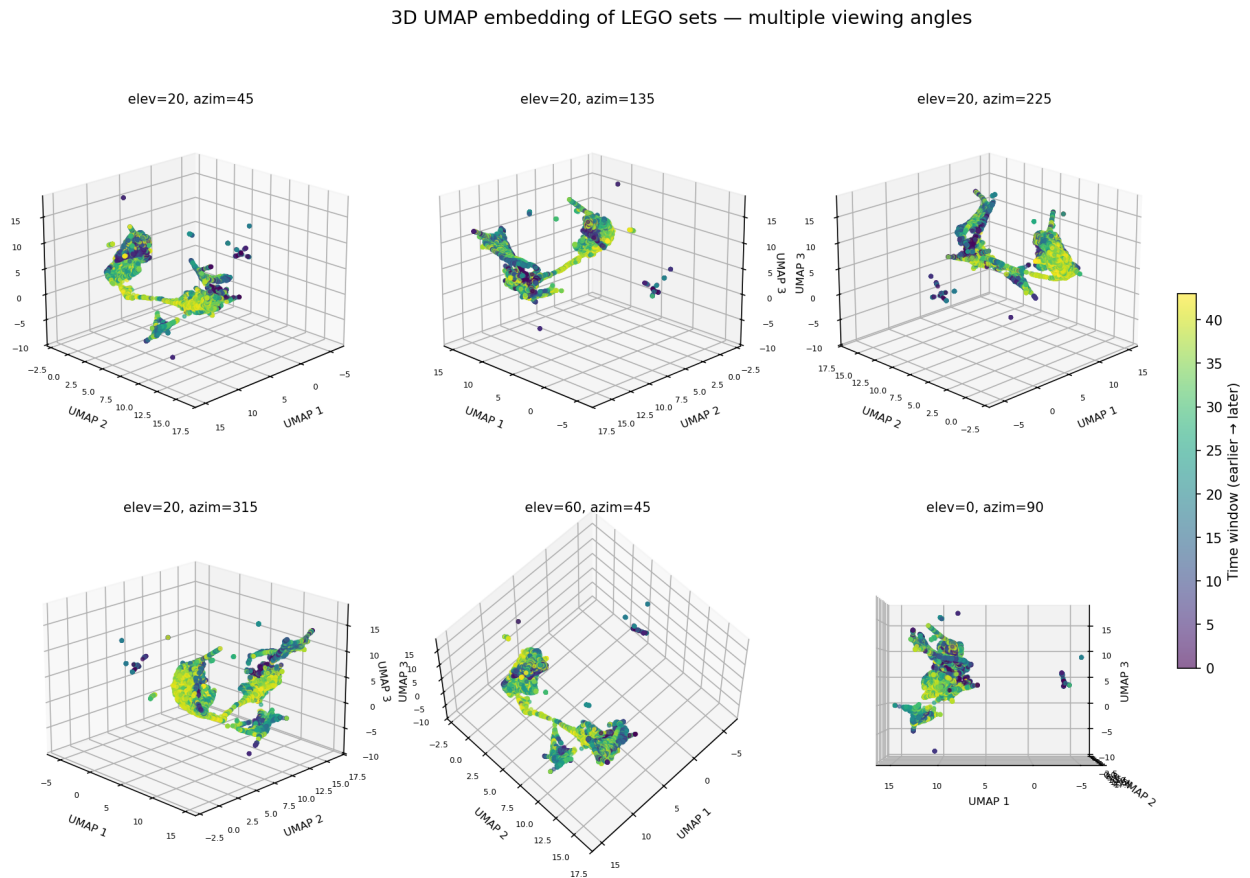


Figure 4: Three-dimensional UMAP embedding of LEGO sets viewed from multiple angles, colored by time window.

References

- [1] Chi, Y., Song, X., Zhou, D., Hino, K., & Tseng, B. L. (2007). Evolutionary spectral clustering by incorporating temporal smoothness. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 153–162). <https://doi.org/10.1145/1281192.1281212>
- [2] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)* (pp. 226–231). AAAI Press.
- [3] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 1–37. <https://doi.org/10.1145/2523813>
- [4] Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97. <https://doi.org/10.1002/nav.3800020109>
- [5] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*. <https://arxiv.org/abs/1802.03426>
- [6] Mishra, A., & Stamp, M. (2025). Cluster analysis and concept drift detection in malware. *arXiv preprint arXiv:2502.14135*. <https://arxiv.org/abs/2502.14135>
- [7] Rebrickable. (2024). LEGO sets database downloads. <https://rebrickable.com/downloads/>
- [8] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)